Matching Types: 404 Syntax Not found

Document number:	P0404R0
Date:	2016-07-11
Project:	ISO/IEC JTC 1/SC 22/WG 21/C++
Working Group:	Evolution
Revises:	None

Reply-to:	Hubert Tong, IBM <hubert.reinterpretcast@gmail.com>
	James Touton <bekenn@gmail.com>

Table of Contents

Introduction

With the recent addition of `constexpr` if statements to C++, we now have a mechanism to express separately-instantiated alternatives based on type properties without adding additional “top-level” templates. However, the checking for said type properties are limited by the use of boolean expressions in series. In this paper, we propose a new construct to perform compile-time selection of alternatives by pattern-matching on (static) types in type, expression, and statement contexts.

The construct is designed to evoke class template partial specialization, and adapts the *template-introduction* syntax from the Concepts TS.

The authors are aware of David Sankel’s paper [P0095R1], and have worked with David on a syntax for his `inspect` construct which, outside of the pattern portion, is similar to what is presented here.

In this paper, `generic` (styled after `_Generic` from C11) is used as a convenience to identify the new construct in code.

Examples

```
// Select on types for types
template <typename T>
struct remove_const {
  using type = typename generic<T>(
    typename{U} const U: U, // introduces U for the purpose of matching against
                            // pattern "const U"
    default: T
  );
};
```

```
// Select for statements
template <typename It>
std::iterator_traits<It>::difference_type distance(It first, It last) {
  generic<It> {
    RandomAccessIterator{U}: return last - first; // introduces U; implied
                                                  // pattern is "U"
    typename{U}: { // same as "default"
      std::iterator_traits<It>::difference_type dist = 0;
      for (It it = first; it != last; ++it, ++dist) { }
      return dist;
```

```
    }
  }
}
```

```
// Select for expressions; result types do not need to be the same
template <typename T>
auto make_pair_or_tuple(T &&...t) {
  return generic<std::integral_constant<size_t, sizeof ...(T)> >(
    std::integral_constant<size_t, 2>:
      std::make_pair(std::forward<T>(t) ...),
    default:
      std::make_tuple(std::forward<T>(t) ...)
  );
}
```

An additional syntax closer to David's `inspect` is also a possibility for future exploration. Notice that the bound names are declared on first use.

```
template <typename T>
constexpr auto array_size_v = inspect<T>(
  [std::array<<!T!>, <!(size_t)N!>>]: N, // typename (see next line) is implied
  [<!typename T!> [<!(auto)N!>]: N
);
```

For the purposes of generating discussion, `inspect` is used as the keyword in the example above. The authors are open to guidance from the committee and opinions from the community.

Motivation and Scope

Like `constexpr` if, the proposed constructs allow for better locality of code. Much of the motivation for this feature is common with that of `constexpr` if, which Ville Voutilainen's paper [P0128R1] explains, and will not be repeated here.

The following demonstrates the reduction of duplicated code and the resulting better reflection of the cohesion inherent to the expressed logic:

Before

```
template <typename T>
struct declared_return_type { };

template <typename R, typename ...Args>
struct declared_return_type<
    R (Args ...)> {
  using type = R;
};

template <typename R, typename ...Args>
struct declared_return_type<
    R (Args ..., ...)> {
  using type = R;
};
```

After

```
template <typename T>
using declared_return_type =
  typename generic<T>(
    <typename R, typename ...Args>
        R (Args ...):
      std::enable_if<true, R>,
    <typename R, typename ...Args>
        R (Args ..., ...):
      std::enable_if<true, R>,
    default:
      std::enable_if<false, void>
  );
```

This paper seeks to compliment `constexpr` if with constructs which implement a "best match" (as opposed to first match) policy. The existing rules for matching specializations of class templates are leveraged, allowing for rather direct mapping from existing code.

Impact On the Standard

For syntactic convenience, the use of a keyword would be useful for this feature. To prevent adding a new keyword, `switch` may be considered, with or without an additional contextual keyword. The authors are aware that some members of the C++ committee have serious reservations on the use of `switch`.

This feature adds a new form of *primary-expression*, a new form of *type-specifier*, and a new form of *selection-statement* to the language. For the cases except the last, further consideration is necessary to examine the impact of allowing the constructs

in contexts which may trigger SFINAE.

These constructs are highly teachable when given a base of knowledge extending to `constexpr` if and class template partial specialization. For those who find class template partial specialization to be arcane, the terser, pattern-like syntax may be more approachable.

The authors believe that this feature has a good amount of synergy with Concepts, and have received confirmation from Andrew Sutton that this proposal does not conflict with the evolution of Concepts Lite.

Design Decisions

This section contains items identified as warranting special note and further discussion. As such, the possibilities presented in this section are not incorporated into later sections of this paper.

Semantics of no match

When there is no match, we propose for the construct to be ill-formed for the type and expression cases. For the statement case, we propose that the construct be treated as having a match on an implied default that selects a null statement.

An alternative for the expression case is to treat the construct as having a match on an implied default that selects `void()`.

Type, value category, etc. properties of the expression form

We propose that the type and value category of the expression form be that of the chosen alternative. The result of the construct is a bit-field if and only if the chosen alternative is a bit-field, and similarly for its being a core constant expression.

Similarly, if the chosen alternative is a *braced-init-list*, then the construct should be treated as that *braced-init-list*.

Introduction syntax for unconstrained non-type template parameters

A form of template introduction, restricted to contexts where it would not be confused with a cast, with a *simple-type-specifier* or *typename-specifier* in place of the *qualified-concept-name* is a natural extension of the introduction syntax.

The authors are aware that additional overloading of similar syntax, which may jeopardize the ability of the language to move in this direction, is being proposed for structured bindings due to dissatisfaction with the use of square brackets.

Multiple patterns selecting the same alternative

It may be desired to be able to have multiple patterns (that are parameterized on the same template parameters) select the same alternative; that is, a syntax for a limited form of fallthrough purely as a shorthand may be useful.

We do not propose adding such a “list” to the set of pack expansion contexts.

Example

```
// Using comma to separate patterns
template <typename T>
struct remove_cv {
  using type = typename generic<T>(
    typename{U} const volatile U, const U, volatile U: U,
    default: T
  );
};
```

Selection of templates

The selection of templates, e.g., as a template template argument, is not proposed by this paper.

Selection on non-type values

Selecting based on non-type values can be effected with the use of `std::integral_constant` (`integral_constant` being a misnomer). Special support need not be added.

Selection on multiple arguments

Selecting on multiple arguments can be effected with the use of `std::tuple`. For simplicity, special support is not presented at this time.

Appearance in signatures

It is an open question whether these constructs should be allowed in signatures or contexts which may directly trigger SFINAE. If they are allowed, a further question —for the case where the selection process is dependent on the arguments of the nearest enclosing template—is whether the selected alternative should be considered to be within the immediate context (much like the substituted *type-id* from a specialization of an alias template).

It is observed that the state of the language is such that “non-dependent” cases of these constructs can be used to redeclare otherwise plainly-declared entities.

```
void f(wchar_t); // #1
void f(char16_t); // #2
template <typename T>
struct A {
  friend void f(typename generic<T>(char16_t: char16_t,
                                    default: wchar_t)); // redeclaration
                                                        // of #1 or #2
};
```

Of note is that the appearance of the proposed constructs in *alias-declaration*s and typedef declarations is an important use case. If the intent is to prevent these constructs from appearing in a signature, then it would be necessary to make dependent cases of said *alias-declarations* and typedef declarations opaque for the purposes of determining signatures.

Placeholder types

Allowing the selection of placeholder types is an idea which could be entertained.

```
template <typename T>
struct A {
  A(const T &);
};

template <typename T>
struct B {
  B(const T &);
};

template <typename Tag, typename T, typename U>
auto f(const T &t, const U &u) {
  return typename generic<Tag>(struct AA: A, struct BB: B, default: auto)(
      std::make_pair(t, u)
      );
}
```

Technical Specifications

Type patterns

A *type-pattern* is either `default`, an optionally-parameterized *type-id*, or a *template-introduction* introducing a single template type parameter. The parameters for a *type-pattern* are to be provided before the *type-id* via either a *template-introduction* or a *template-parameter-list* delimited by angle brackets. The latter case can be constrained by an optional *requires-clause*.

A *template-introduction* with `typename` in place of a *qualified-concept-name* introduces an unconstrained template type parameter or parameter pack for each *identifier* of its *introduction-list*.

When a *type-pattern* contains no *type-id*, then the *type-id* is implied to be the single template type parameter introduced by the pattern. A *type-pattern* of the form `default` corresponds to `typename{`*unique*`}`, where *unique* is a name which is otherwise unbound.

```
// the following type-patterns are equivalent
default
typename{T}
typename{T} T
<class T> T
```

Generic selection constructs

A *generic selection construct* accepts a *type-id* as an argument, and selects the alternative associated with the *type-pattern* that is the best match for the argument. The semantics for determining the best match is equivalent, except for the treatment of the primary template, to that of selecting the definition to be used for a template specialization.

The primary template has the form

```
template <typename T> struct unique { };
```

and each alternative is a definition of either an explicit or partial specialization of said primary template where the *template-argument-list* of the *simple-template-id* consists of the (possibly implicit) *type-id* from the associated *type-pattern*. Selecting the primary template is considered to be a “no match” situation, and a partial specialization whose argument list is identical to the implicit argument list of the primary template is considered to be more specialized in this context.

If there is no best match, the generic selection construct is ill-formed.

If the set of definitions would be ill-formed, e.g., because of multiple definitions of the same entity, then the generic selection construct is ill-formed.

A generic selection expression is type-dependent if its argument or any of its *type-pattern*s is type-dependent; otherwise, it is type-dependent or value dependent if and only if its selected alternative is so. Similarly, *mutatis mutandis*, for a generic selection type being a dependent type.

Validity of alternatives

Unlike `constexpr` if, alternatives in a generic selection construct can remain dependent (on the arguments to be deduced for the parameters in its pattern) and not instantiated.

If an alternative is unselected and at all dependent (even if only on arguments to an enclosing template), the implementation shall not instantiate it; otherwise, the alternative shall be well-formed. If there is no valid instantiation of an unselected alternative, the program is ill-formed; no diagnostic required.

Syntax

In every form of generic selection statement, the alternative is separated from the *type-pattern* by a colon. In the type form, the alternative is a *type-id*. In the expression form, the alternative is an *initializer-clause*. In the statement form, the

alternative is a *statement*; if it is not a *compound-statement*, it is as if it was rewritten to be a compound-statement containing the original alternative.

In the type and expression forms, comma is used to separate alternatives from the following *type-pattern*. In the statement form, no such separator is necessary; the alternative is either delimited by braces or ends in a semicolon.

The type form begins with the keyword `typename`. In the type and expression forms, parentheses are used to delimit the set of alternatives; braces are used for the statement form.

References

[P0095R1] Sankel, D. “Pattern Matching and Language Variants” <http://wg21.link/p0095r1>

[P0128R1] Voutilainen, V. “constexpr_if” <http://wg21.link/p0128r1>

Acknowledgments

The authors would like to thank David Sankel and Andrew Sutton—and any others who may have been unintentionally missed—for their feedback and encouragement. Any mistakes in this paper are the responsibility of the authors.

Appendix

Type list example

```
template <typename ...> struct type_list { };

template <typename List>
using list_head = typename generic<List>(
  <typename Elem, typename ...Elems> type_list<Elem, Elems ...>: Elem
);

template <typename List>
using list_tail = typename generic<List>(
  <typename Elem, typename ...Elems>
      type_list<Elem, Elems ...>: type_list<Elems ...>
);
```

Example of copying top-level *cv*-qualification

```
template <typename T, typename U>
struct preserve_cv {
  using type = typename generic<T>(
    typename{V} const V: const std::remove_cv_t<U>,
    typename{V} volatile V: volatile std::remove_cv_t<U>,
    typename{V} const volatile V: const volatile U,
    default: std::remove_cv_t<U>
  );
};

template <typename T, typename U>
using preserve_cv_t = typename preserve_cv<T, U>::type;
```